

6.2. Entorno Tecnológico y Estándares de Datos

El proyecto adopta software libre y estándares abiertos (OGC, ISO) en toda la cadena de procesamiento, publicación y consumo de datos geoespaciales. Esta decisión garantiza accesibilidad sin costos de licenciamiento, sostenibilidad a largo plazo y transferibilidad del modelo a otros convenios o entidades con infraestructura similar. En la práctica, el stack es 100% JavaScript/TypeScript sobre Node.js con persistencia espacial nativa en PostgreSQL/PostGIS, lo que elimina la necesidad de componentes Java/.NET externos y reduce la superficie operativa.

6.2.1. Herramientas de procesamiento y análisis geoespacial

La herramienta principal de procesamiento es **PostGIS 3.4** sobre **PostgreSQL 16** (licencia PostgreSQL, ambos open source), motor de base de datos que almacena las geometrías como columnas nativas y soporta índices GiST, operadores espaciales (`ST_Intersects`, `ST_DWithin`, `ST_AsGeoJSON`) y funciones geodésicas (`geography` con `ST_Distance` en metros). Toda la lógica de negocio geoespacial se ejecuta en SQL, no en la capa de aplicación, lo que evita la serialización de geometrías hacia/desde el backend Node.js.

Para la carga inicial de capas masivas (departamentos, municipios, hidrografía) se utiliza **shpjs 4.0.4** (licencia BSD-3-Clause) que parsea Shapefiles en el navegador y los inserta vía SQL batch. Para la validación y composición cartográfica previa al despliegue se utiliza **QGIS 3.40 LTS** (licencia GPL v2) en la fase de preparación, no en runtime.

6.2.2. Formatos y estándares de datos

El almacenamiento canónico es **PostgreSQL 16 + PostGIS 3.4** (image Docker `postgis/postgis:16-3.4`), con la geometría viviendo en columnas nativas (`geometry(Point, 4326)`, `geometry(MultiPolygon, 4686)`, etc.) y siguiendo el estándar ISO 19136 (GML/SFA). Los SRID del proyecto son 4326 (WGS 84, propuestas) y 4686 (MAGNA-SIRGAS, predios) — **SRID mixto real**, no unificado.

Para exportación e intercambio con entidades externas se admite **Shapefile** (`.shp`, dominio público, vía `shpjs` o `shp2pgsql`) y se generan reportes en **CSV UTF-8 con BOM y separador ;** (codificación `es-c0` para Excel-en-español, RFC 4180 con quoting). Para la web se sirve **GeoJSON** vía API routes (`ST_AsGeoJSON` desde SQL) y se renderiza directamente sobre Leaflet sin servicios WMS/WFS intermediarios.

Nota: GeoServer y QGIS Server aparecen en el modelo BDG original como alternativas evaluadas, pero **no se utilizan en el runtime actual**. La publicacion de capas para la web se realiza directamente desde Next.js API routes que devuelven GeoJSON, simplificando la arquitectura y eliminando un componente operativo adicional.

6.2.3. Herramientas para el visor web

La publicacion web se realiza integramente con **Next.js 15.1.6** (licencia MIT) + **React 19.0.0** (licencia MIT) + **TypeScript 5.7.3** (licencia Apache-2.0). El backend es el mismo proceso Node.js de Next.js (no hay servidor de mapas dedicado): las API routes (/api/reportes, /api/analisis/buffer, /api/interventions/import) ejecutan SQL parametrizado sobre postgres-js y devuelven JSON o GeoJSON directamente al cliente.

En el cliente se utiliza **Leaflet 1.9.4** (licencia BSD-2-Clause) con **React-Leaflet 5.0.0** (licencia Hippocratic 2.1). La justificacion: peso reducido (~40 KB minificado), compatibilidad universal, plugins OGC disponibles si en el futuro se quisiera migrar a WMS/WFS sin reescribir el frontend, y binding oficial para React que evita sincronizar manualmente refs y props.

La autenticacion se gestiona con **NextAuth 5.0.0-beta.25** (licencia MIT) sobre JWT strategy, con **bcryptjs 2.4.3** (puro JS, sin compilacion nativa) para el hash de contraseñas. La exportacion CSV usa una libreria propia lib/csv.ts (~50 lineas) sin dependencias externas.

Diagrama 1 — Estructura del repositorio TerraSight

```
TG-Nikoll/  
|-- platform/ (raiz del codigo)  
|   |-- src/  
|       |-- app/    (Next.js App Router)  
|       |   |-- admin/  
|       |   |-- analisis/  
|       |   |-- catalogos/ + 5 sub-CRUD  
|       |   |-- intervenciones/  
|       |   |-- predios/  
|       |   |-- quebradas/  
|       |   |-- mapa/    (visor)  
|       |   |-- monitoreo/  
|       |   |-- reportes/  
|       |-- components/  
|       |   |-- ui/      (primitives)  
|       |   |-- map/     (leaflet + compass)  
|       |   |-- layout/  (sidebar + topbar)  
|       |   |-- dashboard/ (charts)  
|       |-- lib/        (business logic)
```

```

| |      |-- repository.ts  (2.4k lineas)
| |      |-- db.ts + auth.ts
| |      `-- csv.ts + validation.ts
| |-- scripts/
| |   |-- db/init/  (9 migraciones SQL 00-09)
| |   |-- db-migrate.ps1
| |   `-- seed.ps1 + setup.ps1
| |-- tests/
| |   |-- unit/  (156 tests vitest)
| |   |-- components/  (26 tests RTL)
| |   `-- e2e/  (10 specs Playwright)
| |-- docs/ internas
| |-- public/ assets
| |-- docker-compose.yml
| `-- package.json
|-- DOCS/  (PRD Nikoll + modelo BDG)
|-- Stich/  (mockups)
|-- .github/workflows/ci.yml
`-- README.md + LICENSE

```

Diagrama 2 — Stack tecnologico del visor

```

+-----+
| USUARIO                                     |
| Analista / Gestor / Administrador         |
| Chrome / Firefox / Edge                   |
+-----+
|
| v HTTPS / JSON / GeoJSON
|
+-----+
| CLIENTE                                     |
| Next.js 15.1.6 + React 19.0.0             |
| React-Leaflet 5.0.0 + Leaflet 1.9.4       |
| TypeScript 5.7.3 + Tailwind 4             |
| recharts 2.15 + @turf/turf 7.2            |
+-----+
|
| v API routes / NextAuth
|
+-----+
| SERVIDOR  (Node.js 22)                    |
| Next.js 15.1.6 Route Handlers + Server Actions |
| next-auth 5.0.0-beta.25  (JWT strategy)    |
| bcryptjs 2.4.3                             |
| Vitest 4.1.10 (156 unit tests)             |
| @playwright/test 1.61.1 (10 e2e tests)     |
+-----+
|
| v postgres-js 3.4.5 (SQL parametrizado)

```

```

+-----+
| DATOS                                     |
| PostgreSQL 16 + PostGIS 3.4             |
| Docker: postgis/postgis:16-3.4         |
| SRID 4326 (propuestas) + 4686 (predios) |
| 9 migraciones SQL idempotentes (00 a 09)|
| Export: GeoJSON (ST_AsGeoJSON) / CSV UTF-8 con BOM |
+-----+

```

Tabla resumen de componentes y versiones

Componente	Version	Licencia	Funcion
Next.js	15.1.6	MIT	Framework full-stack (App Router + API)
React	19.0.0	MIT	UI library
TypeScript	5.7.3	Apache-2.0	Type-safety en todo el codebase
Tailwind CSS	4.0.0	MIT	Estilos utility-first
Leaflet	1.9.4	BSD-2-Clause	Renderer de mapas en cliente
React-Leaflet	5.0.0	Hippocratic 2.1	Binding React para Leaflet
NextAuth	5.0.0-beta.25	MIT	Auth con JWT + Credentials
bcryptjs	2.4.3	MIT	Hash de contraseñas (puro JS)
postgres (postgres-js)	3.4.5	Unlicense	Cliente PostgreSQL/PostGIS
@turf/turf	7.2.0	MIT	Análisis espacial client-side
recharts	2.15.0	MIT	Graficos del dashboard
shpjs	4.0.4	BSD-3-Clause	Parser de Shapefiles en navegador
@radix-ui/react-*	1.x / 2.x	MIT	Primitivos accesibles
lucide-react	0.469.0	ISC	Iconografía
Vitest	4.1.10	MIT	Test runner unitario
@testing-library/react	16.3.2	MIT	Tests de componentes
@playwright/test	1.61.1	Apache-2.0	Tests e2e
happy-dom	20.11.0	MIT	DOM polyfill para Vitest

Componente	Version	Licencia	Funcion
PostgreSQL + PostGIS	16 + 3.4	PostgreSQL License	Almacen y analisis espacial
Docker image (BD)	postgis/postgis:16-3.4	PostgreSQL License	Runtime de la BD
QGIS Desktop	3.40 LTS	GPL v2	Preparacion de datos (fuera del runtime)

Plantilla generada para la subseccion 6.2 del documento del convenio. Versiones verificadas al 2026-07-20 contra package.json y docker-compose.yml. Para convertir este HTML a DOCX: usar VSCode + extension "Markdown PDF" o pandoc. Para editar: abrir el archivo .html en cualquier editor.